

Software Requirements Specification

Project: S32K Media Player

Architecture Team

Ngày 10 tháng 2 năm 2026

Mục lục

1	Giới thiệu (Introduction)	2
1.1	Mục đích (Purpose)	2
1.2	Phạm vi (Scope)	2
1.3	Định nghĩa và Viết tắt (Definitions and Abbreviations)	2
2	Mô tả tổng quan (Overall Description)	3
2.1	Viễn cảnh sản phẩm (Product Perspective)	3
2.2	Tính năng sản phẩm (Product Functions)	3
2.3	Đặc điểm người dùng (User Characteristics)	3
3	Các tính năng hệ thống (System Features)	4
3.1	Core Playback Logic	4
3.1.1	Description	4
3.1.2	Functional Requirements	4
3.2	Hardware Communication (Serial Manager)	4
3.2.1	Description	4
3.2.2	Functional Requirements	4
3.3	Library & Storage Management	4
3.3.1	Description	4
3.3.2	Functional Requirements	4
4	Đặc tả Use Case (Use Case Specification)	5
4.1	UC1: Play/Pause Music	5
4.2	UC2: Load Track	6
4.3	UC3: Adjust Volume (UI)	7
4.4	UC4: Adjust Volume (Board)	7
4.5	UC5: Control Playback (Board)	8
4.6	UC6: Shuffle/Repeat	8
4.7	UC7: Manage Playlist	8
4.8	UC8: Load from USB	9
4.9	UC9: Connect Serial Port	9
5	Yêu cầu phi chức năng (Non-functional Requirements)	10
5.1	Performance	10
5.2	Reliability	10
5.3	Portability	10

1 Giới thiệu (Introduction)

1.1 Mục đích (Purpose)

Tài liệu này mô tả các yêu cầu phần mềm cho dự án **S32K Media Player**. Hệ thống là một ứng dụng nghe nhạc chạy trên nền tảng Linux, được thiết kế theo kiến trúc Model-View-Controller (MVC) và tích hợp khả năng điều khiển thông qua phần cứng ngoại vi (S32K144 Board) qua giao thức Serial/UART.

1.2 Phạm vi (Scope)

Sản phẩm phần mềm là một ứng dụng Desktop với các tính năng chính:

- Phát nhạc (Play, Pause, Next, Previous, Seek).
- Quản lý thư viện nhạc và danh sách phát (Playlist).
- Điều chỉnh âm lượng qua giao diện phần mềm và nút vật lý phần cứng.
- Hiển thị thông tin bài hát (Metadata) và ảnh bìa (Cover Art).
- Giao tiếp hai chiều với S32K Board để đồng bộ trạng thái và nhận lệnh điều khiển.

1.3 Định nghĩa và Viết tắt (Definitions and Abbreviations)

Term	Definition
MVC	Model-View-Controller Architecture
SDL	Simple DirectMedia Layer (Audio & Windowing backend)
ImGui	Dear ImGui (GUI Library)
UART	Universal Asynchronous Receiver-Transmitter
S32K	NXP S32K144 Microcontroller Board
Metadata	Thông tin mô tả bài hát (Title, Artist, Album)

2 Mô tả tổng quan (Overall Description)

2.1 Viễn cảnh sản phẩm (Product Perspective)

Hệ thống hoạt động như một ứng dụng độc lập trên Linux, sử dụng:

- **View Layer:** ImGui để hiển thị giao diện người dùng.
- **Controller Layer:** Điều phối logic chính ('AppController') và các module con ('AudioPlayer', 'Serial-Manager').
- **Model Layer:** Quản lý trạng thái ('PlayerState') và dữ liệu bài hát ('MediaFile').
- **Hardware Interface:** Kết nối qua cổng Serial (/dev/tty*) với S32K Board.

2.2 Tính năng sản phẩm (Product Functions)

Các tính năng chính được chia thành các nhóm:

1. **Playback:** Điều khiển phát nhạc cơ bản.
2. **Library Management:** Quét và quản lý tập tin nhạc.
3. **Hardware Integration:** Kết nối và đồng bộ với S32K.
4. **UI Interaction:** Hiển thị trực quan và phản hồi người dùng.

2.3 Đặc điểm người dùng (User Characteristics)

- Người dùng cuối muốn nghe nhạc trên máy tính Linux.
- Người dùng muốn trải nghiệm điều khiển nhạc bằng phần cứng chuyên dụng (nút vặn volume, nút bấm vật lý).

3 Các tính năng hệ thống (System Features)

3.1 Core Playback Logic

3.1.1 Description

Module chịu trách nhiệm xử lý luồng phát nhạc, bao gồm nạp file, giải mã và điều khiển đầu ra âm thanh.

3.1.2 Functional Requirements

- **REQ-PB-01:** Hệ thống phải hỗ trợ các định dạng file: .mp3, .wav, .flac, .ogg.
- **REQ-PB-02:** Hệ thống phải duy trì máy trạng thái (STOPPED, PLAYING, PAUSED).
- **REQ-PB-03:** Hệ thống phải cho phép tua (seek) đến vị trí bất kỳ trong bài hát.
- **REQ-PB-04:** Hệ thống phải tự động chuyển sang bài tiếp theo khi bài hiện tại kết thúc (nếu có trong Queue).
- **REQ-PB-05:** Tính năng Shuffle phải xáo trộn ngẫu nhiên danh sách phát mà không làm mất bài.
- **REQ-PB-06:** Tính năng Repeat phải hỗ trợ: Repeat One, và Off.

3.2 Hardware Communication (Serial Manager)

3.2.1 Description

Module quản lý kết nối Serial/UART với S32K Board để gửi/nhận lệnh.

3.2.2 Functional Requirements

- **REQ-HW-01:** Hệ thống phải liệt kê được các cổng Serial khả dụng trên máy tính.
- **REQ-HW-02:** Hệ thống phải thiết lập kết nối với thông số: Baudrate 9600, 8N1.
- **REQ-HW-03:** Hệ thống phải nhận bản tin điều khiển Volume ('VR: value') và cập nhật âm lượng ứng dụng.
- **REQ-HW-04:** Hệ thống phải nhận bản tin điều khiển Playback ('cmd: play/pause/next') và thực thi lệnh tương ứng.

3.3 Library & Storage Management

3.3.1 Description

Module quét và quản lý thư viện nhạc từ ổ cứng hoặc USB.

3.3.2 Functional Requirements

- **REQ-LB-01:** Hệ thống phải quét đệ quy thư mục được chỉ định để tìm file nhạc hợp lệ.
- **REQ-LB-02:** Hệ thống phải trích xuất Metadata (Title, Artist, Album) sử dụng TagLib.
- **REQ-LB-03:** Hệ thống phải trích xuất ảnh bìa (Cover Art) nhúng trong file nhạc.
- **REQ-LB-04:** Hệ thống phải hỗ trợ tải nhạc từ thiết bị lưu trữ ngoài (USB) khi được mount.

4 Đặc tả Use Case (Use Case Specification)

4.1 UC1: Play/Pause Music

Pre-conditions	• Track đã được load (UC2). • Audio system đã được khởi tạo. • Đối với Pause: Music đang phát (PlayerState = PLAYING).
Trigger	Người dùng nhấn nút Play/Pause trên giao diện hoặc Board gửi tín hiệu điều khiển.
Main Success Scenario	(Basic Flow - Play) 1. Actor thực hiện hành động (Click nút Play hoặc gửi lệnh). 2. View gọi <code>AppController.play()</code>. 3. <code>PlaybackController</code> gọi <code>AudioPlayer.play()</code>. 4. <code>AudioPlaybackImpl</code> gọi <code>Mix_ResumeMusic()</code>. 5. <code>PlayerState</code> cập nhật trạng thái thành PLAYING. 6. UI cập nhật icon Play thành Pause.
Alternative Flows	A1. Chưa load track 1. Hệ thống hiển thị thông báo lỗi trên UI. 2. Không thực hiện hành động phát nhạc.
Post-conditions	• Trạng thái phát nhạc thay đổi (Playing ↔ Paused). • <code>PlayerState</code> được cập nhật. • UI hiển thị trạng thái tương ứng.

4.2 UC2: Load Track

Pre-conditions	• Library đã có tracks. • UI đang hiển thị danh sách nhạc.
Trigger	Người dùng click vào một bài hát trong danh sách.
Main Success Scenario	(Basic Flow) 1. User click vào track trong danh sách nhạc. 2. View gọi <code>AppController.loadTrack(filePath)</code>. 3. <code>PlaybackController</code> tạo đối tượng <code>MediaFile</code>. 4. <code>MediaFile</code> trích xuất Metadata sử dụng <code>TagLib</code>. 5. <code>AudioLoaderImpl</code> load file sử dụng <code>Mix_LoadMUS()</code>. 6. <code>PlayerState</code> cập nhật <code>currentTrack</code>. 7. UI hiển thị thông tin bài hát tại khu vực Now Playing.
Alternative Flows	A1. File không tồn tại 1. <code>AudioLoader</code> trả về lỗi. 2. Hiển thị thông báo lỗi cho User. A2. Định dạng file không hỗ trợ 1. <code>Mix_LoadMUS()</code> thất bại. 2. Log error và thông báo cho User.
Post-conditions	• Track được load vào <code>AudioPlayer</code>. • Metadata hiển thị trên UI. • Hệ thống sẵn sàng để phát.

4.3 UC3: Adjust Volume (UI)

Pre-conditions	• UI đang hiển thị. • Audio system đã được khởi tạo.
Trigger	Người dùng kéo thanh trượt volume.
Main Success Scenario	(Basic Flow) 1. User kéo slider volume trên PlayerBar. 2. View gọi <code>AppController.setVolume(value)</code>. 3. <code>VolumeController</code> gọi <code>AudioPlayer.setVolume(value)</code>. 4. <code>AudioVolumeImpl</code> gọi <code>Mix_VolumeMusic(sdlValue)</code>. 5. <code>PlayerState.setVolume(value)</code>. 6. UI cập nhật vị trí slider.
Alternative Flows	A1. Giá trị Volume ngoài khoảng (0-100) 1. Hệ thống kẹp (clamp) giá trị về 0 hoặc 100. 2. Tiếp tục thực hiện Main Flow.
Post-conditions	• Giá trị Volume thay đổi. • UI slider hiển thị vị trí mới.

4.4 UC4: Adjust Volume (Board)

Pre-conditions	• Serial connection đã được thiết lập (UC9). • Board đang hoạt động.
Trigger	Người dùng xoay núm volume trên Board.
Main Success Scenario	(Basic Flow) 1. User xoay núm volume trên Board. 2. Board đọc giá trị ADC (0-4095). 3. Board gửi bản tin VR: {value}n qua Serial. 4. <code>SerialIOImpl</code> nhận dữ liệu. 5. <code>BoardCommunicator</code> quy đổi: $volume = adc \times 100 / 4095$. 6. Gọi <code>AppController.setVolume(percent)</code>. 7. UI cập nhật slider tương ứng.
Alternative Flows	A1. Serial bị ngắt kết nối 1. Dữ liệu không được nhận, volume không đổi.
Post-conditions	Volume hệ thống thay đổi đồng bộ với phần cứng.

4.5 UC5: Control Playback (Board)

Pre-conditions	Serial connection OK, Track loaded.
Trigger	Người dùng nhấn nút SW trên Board.
Main Success Scenario	(Basic Flow) 1. User nhấn nút trên Board. 2. Board gửi lệnh <code>cmd:{command}</code> n. 3. <code>BoardCommunicator</code> parse lệnh (play, pause, next). 4. Gọi <code>AppController</code> tương ứng. 5. UI cập nhật trạng thái mới.
Exception Flows	E1. Lệnh không hợp lệ: Bỏ qua và log warning.
Post-conditions	Trạng thái Playback thay đổi.

4.6 UC6: Shuffle/Repeat

Pre-conditions	Có tracks trong Queue.
Trigger	User click nút Shuffle hoặc Repeat.
Main Success Scenario	(Shuffle Enable) 1. User click Shuffle. 2. <code>AppController.toggleShuffle()</code>. 3. <code>PlaylistManager</code> random hóa Queue. 4. UI highlight icon Shuffle.
Alternative Flows	(Repeat Cycle) 1. User click Repeat. 2. Cycle: OFF → ONE → OFF. 3. UI update icon.
Post-conditions	Thứ tự Queue hoặc chế độ Repeat thay đổi.

4.7 UC7: Manage Playlist

Pre-conditions	UI ở tab Playlist.
Trigger	User click Create, Add Track, hoặc Play Playlist.
Main Success Scenario	(Create Playlist) 1. User click "Create Playlist". 2. Nhập tên, mô tả, chọn màu. 3. <code>PlaylistManager.createPlaylist()</code>. 4. Danh sách hiển thị Playlist mới.
Post-conditions	Playlist database được cập nhật.

4.8 UC8: Load from USB

Pre-conditions	USB đã mount.
Trigger	User click Refresh/Load trên Storage Panel.
Main Success Scenario	(Basic Flow) 1. User chọn thiết bị USB, click "Load". 2. <code>AppController.loadFromStorage(path)</code>. 3. <code>PlaylistManager</code> quét đệ quy thư mục. 4. Lọc file .mp3, .wav, .flac. 5. Tạo <code>MediaFile</code>, extract metadata. 6. Thêm vào Library.
Exception Flows	E1. USB ngắt kết nối: Dừng quét, báo lỗi.
Post-conditions	Tracks từ USB được thêm vào Library.

4.9 UC9: Connect Serial Port

Pre-conditions	Cổng Serial khả dụng.
Trigger	User click Connect.
Main Success Scenario	(Basic Flow) 1. User chọn cổng, click "Connect". 2. <code>AppController.connectToPort()</code>. 3. Mở cổng với cờ <code>O_RDWR</code>. 4. Khởi chạy thread đọc tin nhắn. 5. UI hiện "Connected".
Exception Flows	E1. Cổng bận/Lỗi quyền: Báo lỗi UI.
Post-conditions	Kết nối Serial được thiết lập.

5 Yêu cầu phi chức năng (Non-functional Requirements)

5.1 Performance

- Thời gian khởi động ứng dụng dưới 2 giây.
- Độ trễ xử lý lệnh từ Board (Serial) đến khi thực thi dưới 100ms.
- Tải CPU khi phát nhạc không vượt quá 10

5.2 Reliability

- Hệ thống không được crash khi tải file nhạc bị lỗi (corrupt).
- Tự động ngắt kết nối Serial an toàn khi rút thiết bị mà không treo ứng dụng.

5.3 Portability

- Mã nguồn tuân thủ chuẩn C++14/17.
- Sử dụng thư viện đa nền tảng (SDL2, ImGui) để dễ dàng build trên các distro Linux khác nhau.